Complete Variable History :

Index No	Variables types	Declaration Position	Scope	Memory Area	Applicable Modifiers	Accessed by	Lifetime
01	Class Variable OR Static Field	Inside Class	Belongs to class	Method Area	private, protected, public, final, transient and volatile.	Class name OR Object reference	Entire program execution (class is unloaded)
02	Instance Variable OR Non-Static Field	Inside Class	Belongs to an object (non-static)	Heap Area	private, default, protected, public, final, transient and volatile.	Object reference	As long as object lives
03	Local Veriable	Inside Method	Inside a method/block /constructor	Stack Area	Only final modifier is applicable.	Only inside method/block	Until method ends
04	Parameter Variable	Inside Method	Passed to methods/constructors	Stack Area	Only final modifier is applicable.	Passed when Calling method	Until method ends

- ➔ Invalid Modifier for **static field**: abstract, synchronized, native, strictfp and final + volatile.
- ➔ Invalid Modifier for **non-static field**: abstract, synchronized, native and strictfp.
- ➔ Invalid Modifier for **Local Variable**: Only final is allowed.
- ➔ Invalid Modifier for **Parameter Variable**: Only final is allowed.

Constructor :
----

* It is playing a major in object creation OR Object construction that is the reason, It is known as Constructor.

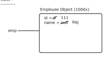
What is the advantage of writing constructor in a class ?

```
public class Employee
{
   private int id;
   private String name;

   public Employee()
   {
   }

   public void set(int id, String name)
   {
     this.id = id;
     this.name = name;
   }
}

   Employee emp = new Employee();  //Variable Initialization with default value
            emp.set(111,"Raj");  //Variable re-initialization with user value
```



* If we don't write constructor in our BLC class then compiler will automatically add one default no argument constructor which will help us to create the object as well as, all our non static fields are initialized with default value using new keyword at the time of object creation.

* So the conclusion is : Our non static variable initialization and re-initialization both are done in different lines
  a) Initialization with default value -> at the time of creating the object
  b) Re-initialization with user value -> at the time of calling the set method

---

If user will define its own constructor :

```
public class Employee
{
   private int id;
   private String name;

   public Employee(int id, String name)
   {
      this.id = id;
      this.name = name;
   }
}


  Employee emp = new Employee(101, "Raj");
```

* In this approach, non static field initialization (using new keyword) and re-initialization (user defined value using constructor) both are done in the **same line, at the time of creating object.**

It is used to construct the object so it is called Constructor.

If the name of the class and name of the method both are exactly same and It should not contain any return type then it is called constructor.

**The main purpose of constructor to initialize the object properties (non sttaic fields) with user-defined value.

Every class must contain at-least one constructor either implicitly added by compiler or explicitly written by user.

Every time we create an object in java by using new keyword, at-least one constructor must be invoked.

A constructor never contain any return type including void also.

Example :

```
package com.ravi.command;

class Student
{
  public void Student() //Method
  {
       System.out.println("I am Method");
  }

  public static void main(String [] args)
  {
    Student s1 = new Student();
    s1.Student();
  }
}
```

A constructor may contain return keyword but not return keyword with value.

```
package com.ravi.command;

class Student
{
  public Student()
  {
       System.out.println("I am Constructor");
       return;  //Valid
  }

  public static void main(String [] args)
  {
    Student s1 = new Student();

  }
}
```

A constructor is automatically called and executed at the time of creating the object.

Types of Constructor :
----

* In Java, We have 3 types of constructors

  1) Default no argument Constructor [Added by Java compiler]
  2) No Argument OR Non Parameterized OR Parameter less Constructor [Written by user]
  3) Parameterized Constructor

Default No Argument Constructor :

Whenever we write a class and if we don't write any type of constructor then automatically one default no argument constructor is added by the compiler.

The access modifier of default constaructor would be same as class access modifier.

It does not accept any parameter.

Example.java

```
public class Example
{

}
```

javac Example.java

Example.class

```
public class Example
{
  public Example() //default constructor
  {
  }
}
```

No Argument OR Non Parameterized OR Parameter less Constructor :
----

* If a developer writes constructor without any parameter then It is called No Argument constructor.

* No argument constructor and default constructor, both look like same the only difference is, default constructor means added by compiler and no argument constructor means written by user.

* The main purpose of this no argument constructor (Written by user) to initialize the non static fields with **default value.**

Example :

```
public class Student
{
   private int id;
   private String name;

   public Student()  //No Argument Constructor
   {
      id = 0;
      name = null;
   }
}
```

Parameterized Constructor :

If we pass one or more parameter to the constructor then it is called parameterized constructor.

By using parameterized constructor all the objects will be initialized with different values.

Example :

```
public class Employee
{
   private int id;
   private String name;

   public Employee(int id, String name)
   {
     this.id = id;
     this.name = name;
   }
}
```

//Program

```
package com.blc;

public class Dog
{
   private String name;
   private String color;
   private double height;
   private int age;

   public Dog(String name, String color, double height, int age)
   {
    super();
    this.name = name;
    this.color = color;
    this.height = height;
    this.age = age;
   }

   @Override
   public String toString()
   {
    return "Dog [name=" + name + ", color=" + color + ", height=" + height + ", age=" + age +
"]";
   }

}
```

```
package com.elc;

import com.blc.Dog;

public class DogDemo
{
    public static void main(String[] args)
    {
        Dog tommy = new Dog("Tommy", "White", 2.5, 3);
        System.out.println(tommy);

    }

}
```